

KNOWTIFY

Plataforma Educativa de Calificación Inteligente

INFORME DE CONSTRUCCIÓN DEL SOFTWARE Backend y Frontend — Énfasis en la Fase de Diseño

Proyecto: KNOWTIFY (nombre técnico interno: edu-grade-mocavi)

Tipo de sistema: Aplicación web full-stack (cliente-servidor)

Documento: Informe del proceso de construcción y diseño

Fecha: 21 de mayo de 2026

Versión del informe: 1.0

Tabla de contenido

Para actualizar la numeración de páginas: haga clic derecho sobre la tabla y seleccione «Actualizar campos» (o seleccione todo y pulse F9).

Actualice este campo para ver la tabla de contenido.

1. Introducción

1.1 Contexto y justificación

KNOWTIFY es una plataforma educativa web desarrollada para resolver una necesidad concreta de las instituciones educativas: la calificación de evaluaciones consume gran parte del tiempo del docente y la información de desempeño llega tarde y de forma poco clara al estudiante. El proyecto integra, en una sola herramienta, tres ejes que normalmente están separados: (1) la calificación masiva de exámenes mediante visión por computador e inteligencia artificial, (2) la gestión de notas por periodo y dimensión (Saber, Hacer, Ser), y (3) la comunicación y seguimiento del desempeño entre docente y estudiante.

Este informe documenta el proceso de construcción del software de extremo a extremo —tanto el backend (servidor y lógica de negocio) como el frontend (interfaz de usuario)— con especial énfasis en la fase de diseño, que es donde se tomaron las decisiones arquitectónicas, de modelo de datos y de experiencia de usuario que sostienen todo el sistema.

1.2 Descripción general del sistema

El sistema se compone de dos grandes aplicaciones que se comunican mediante una API REST sobre HTTP:

- **Backend (FastAPI + PostgreSQL):** expone la lógica de negocio, la persistencia de datos, la autenticación y la integración con los servicios de IA y de visión por computador.
- **Frontend (Next.js + React):** ofrece dos portales diferenciados —Docente y Estudiante— además de un sitio público de presentación.

KNOWTIFY ofrece un portal público (presentación, secciones informativas y suscripción al boletín), un Portal del Docente (gestión de notas, calificador masivo OMR/IA y calendario académico) y un Portal del Estudiante (tablero de desempeño, detalle por periodo y mensajería con el docente).

2. Objetivos

2.1 Objetivo general

Construir una plataforma web que automatice la calificación de evaluaciones y centralice la gestión académica de notas, comunicación y seguimiento del

desempeño, reduciendo el tiempo operativo del docente y mejorando la retroalimentación al estudiante.

2.2 Objetivos específicos

1. Diseñar una arquitectura cliente-servidor desacoplada mediante una API REST.
2. Modelar la información académica (estudiantes, notas por periodo y dimensión, exámenes y resultados).
3. Implementar un motor de calificación óptica de marcas (OMR) con OpenCV para leer hojas de respuesta.
4. Identificar automáticamente al estudiante a partir del nombre manuscrito usando IA de visión.
5. Integrar un modelo de IA generativa (Gemini) para generar exámenes y asistir al estudiante.
6. Garantizar la seguridad mediante autenticación JWT, cifrado de contraseñas y limitación de tasa.
7. Diseñar una interfaz moderna, accesible y responsive con ayuda contextual integrada.

3. Alcance del sistema

El sistema cubre el ciclo completo de calificación y seguimiento académico de un grupo-clase. A continuación se detallan las funcionalidades incluidas, organizadas por módulo.

Módulo	Funcionalidades incluidas
Sitio público	Landing, secciones (Sobre nosotros, Cursos, Precios, Guía, Contacto), demo animada y suscripción al boletín.
Autenticación	Inicio de sesión de docente (usuario/contraseña + JWT) y de estudiante (documento).
Gestión de notas	Registro por dimensión (Saber/Hacer/Ser), cálculo de nota final y nivel de desempeño por periodo.
Calificador masivo	Lector OMR (OpenCV) e IA (Gemini); identificación del estudiante por nombre; guardado por periodo.
Exámenes con IA	Generación de exámenes de opción múltiple

	y su clave de respuestas.
Calendario	Eventos académicos (evaluaciones, actividades) por grado y grupo.
Portal estudiante	Tablero de desempeño, gráfica de evolución, detalle por periodo y mensajería.
Mensajería	Comunicación docente–estudiante con conteo de no leídos.

Tabla 1. Alcance funcional del sistema por módulo.

4. Metodología de desarrollo

El desarrollo siguió un modelo incremental e iterativo: el producto se construyó por incrementos funcionales, partiendo de una base estable y agregando módulos verificables en cada iteración. Este enfoque permitió validar cada funcionalidad de forma temprana y mantener un sistema siempre ejecutable.

Fase	Actividades principales	Productos
Análisis	Identificación de requisitos, actores y casos de uso.	Requisitos funcionales y no funcionales.
Diseño	Arquitectura, modelo de datos, API, UI/UX, motor OMR, seguridad.	Diagramas y especificaciones.
Desarrollo	Codificación del backend y del frontend por incrementos.	Código fuente versionado.
Pruebas	Verificación funcional y de extremo a extremo (E2E).	Evidencias de pruebas.
Despliegue	Puesta en marcha de servidores y base de datos.	Sistema en ejecución.

Tabla 2. Fases de la metodología incremental aplicada.

5. Fase de Análisis

5.1 Actores del sistema

Actor	Descripción	Acceso
Visitante	Usuario no autenticado que	Público

	explora el sitio público.	
Docente	Gestiona notas, califica exámenes y administra el calendario.	JWT (rol teacher)
Estudiante	Consulta su desempeño y se comunica con el docente.	JWT (rol student)
Servicios de IA	Gemini (generación/visión) como actor externo del sistema.	API externa

Tabla 3. Actores que interactúan con el sistema.

5.2 Requisitos funcionales

- RF-01: El docente debe autenticarse con usuario y contraseña.
- RF-02: El estudiante debe autenticarse con su número de documento.
- RF-03: El sistema debe registrar y calcular notas por dimensión y periodo.
- RF-04: El sistema debe calificar hojas de respuesta mediante lectura óptica (OMR).
- RF-05: El sistema debe identificar al estudiante por su nombre escrito en la hoja.
- RF-06: El docente debe poder elegir el periodo donde se guarda el examen calificado.
- RF-07: El sistema debe generar exámenes de opción múltiple con IA.
- RF-08: El estudiante debe visualizar su evolución y el detalle de notas por periodo.
- RF-09: El sistema debe permitir mensajería entre docente y estudiante.
- RF-10: El sistema debe gestionar un calendario de eventos académicos.
- RF-11: El sistema debe permitir exportar las notas a Excel y PDF.
- RF-12: El sitio público debe permitir la suscripción al boletín.

5.3 Requisitos no funcionales

Atributo	Especificación
Seguridad	Autenticación con JWT, contraseñas cifradas con bcrypt y limitación de tasa (rate limiting).
Rendimiento	La lectura OMR procesa cada hoja localmente en ~30 ms; respuestas comprimidas con GZip.
Usabilidad	Interfaz responsive, modo claro/oscuro y ayuda contextual integrada (HelpTip).

Mantenibilidad	Separación por capas y cliente HTTP central único en el frontend.
Disponibilidad	Arquitectura desacoplada que permite escalar backend y frontend por separado.

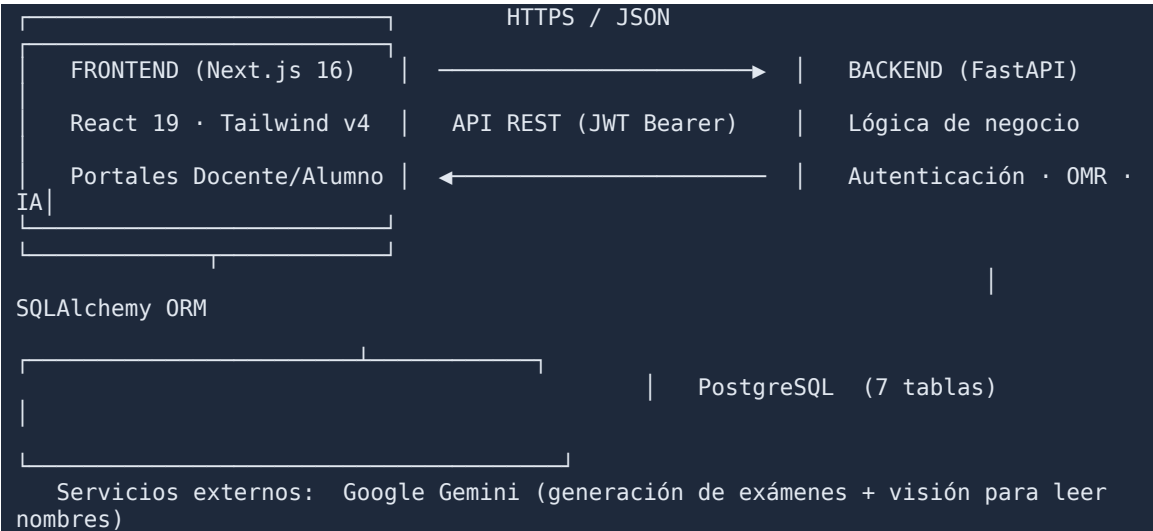
Tabla 4. Requisitos no funcionales del sistema.

6. Fase de Diseño

La fase de diseño es el núcleo de este informe. En ella se definieron la arquitectura, el modelo de datos, el contrato de la API, el diseño de la interfaz y los algoritmos clave (motor OMR e identificación por IA). Las decisiones aquí tomadas determinan la calidad, la seguridad y la mantenibilidad de todo el software.

6.1 Arquitectura general

Se adoptó una arquitectura cliente-servidor de tres capas, con separación estricta entre la presentación (frontend), la lógica de negocio (backend) y la persistencia (base de datos). La comunicación entre cliente y servidor se realiza exclusivamente mediante una API REST en formato JSON, autenticada con tokens JWT.



Esta separación permite que el frontend y el backend evolucionen y se desplieguen de forma independiente, y que la lógica sensible (validación, seguridad, IA) resida siempre en el servidor, nunca en el navegador.

6.2 Diseño del modelo de datos

El modelo de datos se diseñó alrededor de siete entidades persistentes implementadas con el ORM SQLAlchemy sobre PostgreSQL. Se emplean identificadores UUID como clave primaria y tipos JSONB para almacenar estructuras flexibles (respuestas y claves de examen).

Entidad (tabla)	Propósito	Campos clave
Student (students)	Estudiantes inscritos.	full_name, document_id, email, grade, grupo

User (users)	Docentes/administradores.	username, hashed_password, full_name
Grade (grades)	Notas por periodo y dimensión.	period_id, score_saber/hacer/ser, final_period_score, performance_level
Exam (exams)	Exámenes creados/calificados.	title, subject, grade, grupo, period_id, answer_key (JSONB)
ExamResult (exam_results)	Resultado de un estudiante en un examen.	exam_id, student_id, answers (JSONB), score
Message (messages)	Mensajería docente-estudiante.	sender/receiver, content, is_read
CalendarEvent (calendar_events)	Eventos del calendario.	title, event_date, event_type, grade, grupo
NewsletterSubscriber	Suscriptores del boletín.	email (único), is_active

Tabla 5. Entidades del modelo de datos y sus campos principales.

Ejemplo de definición de una entidad con SQLAlchemy (entidad Grade, que almacena las notas por dimensión y la nota final calculada):

```
class Grade(Base):
    __tablename__ = "grades"
    id = Column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)
    enrollment_id = Column(UUID(as_uuid=True), nullable=False, index=True)
    period_id = Column(Integer, nullable=False, index=True)
    score_saber = Column(Numeric(3, 2), default=1.0)
    score_hacer = Column(Numeric(3, 2), default=1.0)
    score_ser = Column(Numeric(3, 2), default=1.0)
    final_period_score = Column(Numeric(3, 2))
    performance_level = Column(String) # SUPERIOR | ALTO | BÁSICO | BAJO
    updated_at = Column(DateTime, server_default=func.now())
```

6.3 Diseño de la API REST

El contrato de la API se diseñó siguiendo principios REST: rutas orientadas a recursos, verbos HTTP semánticos (GET/POST/PUT/DELETE) y respuestas JSON validadas con Pydantic. La API expone alrededor de 30 endpoints. La siguiente tabla resume los más representativos.

Método y ruta	Función
POST /login/teacher	Autenticación del docente (devuelve JWT).
POST /login/student	Autenticación del estudiante por documento.

GET /grades/	Lista de notas por grado/grupo/periodo.
POST /grades/	Registro/actualización de notas.
GET /answer-sheet	Genera la hoja de respuestas imprimible (PDF).
POST /upload-exams-omr	Califica hojas con OMR e identifica al estudiante por nombre.
POST /grades/apply-omr	Guarda las notas del OMR en una dimensión y periodo.
POST /ai/generate-exam	Genera un examen de opción múltiple con IA.
GET /students/{id}/exam-results	Resultados de exámenes de un estudiante (por periodo).
POST /chatbot	Asistente de IA (tutor) para el estudiante.
POST /messages/ · GET /messages/	Mensajería docente–estudiante.
GET/POST/PUT/DELETE /events/	Gestión del calendario académico.
POST /newsletter	Suscripción al boletín (público).

Tabla 6. Endpoints representativos de la API REST.

6.4 Diseño de seguridad

La seguridad se diseñó como una preocupación transversal. Las decisiones principales fueron:

- **JWT:** autenticación sin estado mediante tokens firmados (HS256) con expiración.
- **bcrypt:** las contraseñas nunca se almacenan en claro; se cifran con bcrypt (12 rondas de sal).
- **Rate limiting:** limitación de tasa por IP (slowapi) en los endpoints sensibles (login 5/min, IA 10–12/min).
- **Validación:** validación de entrada con Pydantic (tipos, longitudes, formato de correo).
- **Defensa:** mensajes de error que no revelan si el usuario existe; logout automático ante 401.

Cifrado de contraseñas y emisión del token (módulo auth):

```
def hash_password(password: str) -> str:
    salt = bcrypt.gensalt(rounds=12)
    return bcrypt.hashpw(password.encode('utf-8'), salt).decode('utf-8')
```

```
def create_access_token(data: dict, expires_delta: Optional[timedelta] = None) -> str:
    to_encode = data.copy()
    expire = datetime.utcnow() + (expires_delta or
timedelta(minutes=JWT_EXPIRATION_MINUTES))
    to_encode.update({"exp": expire})
    return jwt.encode(to_encode, JWT_SECRET_KEY, algorithm=JWT_ALGORITHM)
```

6.5 Diseño del motor OMR (visión por computador)

El módulo de calificación óptica de marcas (OMR) se diseñó para leer hojas de respuesta de forma rápida, offline y sin coste por hoja, usando OpenCV. El diseño se basa en una hoja patrón con cuatro cuadrados fiduciales en las esquinas que permiten corregir la perspectiva de una foto y normalizar la imagen a un lienzo fijo (1000×1414 px, proporción A4).

Flujo de diseño del motor OMR:

8. Decodificar la imagen o renderizar la página del PDF.
9. Detectar los cuatro fiduciales y corregir la perspectiva (warp) a un lienzo normalizado.
10. Recortar la franja del nombre para identificar al estudiante mediante IA de visión.
11. Medir el porcentaje de relleno de cada burbuja A/B/C/D.
12. Comparar con la clave y calcular la nota en escala 1.0–5.0.

La geometría de la hoja es la única fuente de verdad: el mismo archivo genera la plantilla imprimible y lee las marcas, de modo que plantilla y lector nunca se desincronizan.

```
# Geometría del lienzo normalizado (proporción A4 vertical)
SHEET_W, SHEET_H = 1000, 1414
MARGIN, FIDUCIAL = 55, 46          # cuadrados de las esquinas
NAME_X0, NAME_X1 = 150, 900       # franja del nombre (se recorta para la IA)
NAME_TOP, NAME_BOT = 198, 280
GRID_TOP, ROW_H, BUBBLE_R = 380, 50, 16  # cuadrícula de respuestas

def name_region(warped):
    """Recorta la franja del NOMBRE y la devuelve como PNG para la IA de
visión."""
    crop = warped[NAME_TOP+4:NAME_BOT-4, NAME_X0+4:NAME_X1-4]
    ok, png = cv2.imencode(".png", crop)
    return png.tobytes() if ok else b""

# Cálculo de la nota a partir de aciertos (escala 1.0 - 5.0)
score = round(correct / num_q * 4 + 1, 2)
```

La hoja de respuestas generada por el sistema (con sus fiduciales, la caja del nombre y la cuadrícula de respuestas) se muestra en la sección de resultados.

6.6 Diseño de la identificación por nombre con IA

Una decisión de diseño relevante fue cómo asignar cada hoja a su estudiante sin pérdida de tiempo. Tras evaluar alternativas (selección manual de una lista o una rejilla de número de documento), se optó por reconocer el nombre manuscrito mediante IA de visión (Gemini). El recorte del nombre se envía junto con la lista exacta de la clase, y el modelo devuelve la coincidencia. El emparejamiento final normaliza acentos y mayúsculas; si no hay coincidencia clara, la hoja queda para asignación manual (respaldo). Este diseño combina automatización (cero clics en el caso común) con robustez (la IA elige de una lista cerrada, no texto libre).

6.7 Diseño de la interfaz (UI/UX)

La interfaz se diseñó con un sistema visual coherente basado en Tailwind CSS v4. Las decisiones de diseño visual y de experiencia fueron:

- **Identidad:** violeta/índigo como colores de marca, con tarjetas redondeadas y sombras suaves.
- **Temas:** modo claro y oscuro conmutables (next-themes), respetando el contraste.
- **Portales:** dos portales claramente diferenciados por color (docente índigo, estudiante violeta).
- **Ayuda:** ayuda contextual integrada (componente HelpTip) junto a cada control complejo.
- **Datos:** visualización de datos con gráficas (recharts) para la evolución del promedio.
- **Responsive:** diseño responsive y animaciones sutiles para reforzar la jerarquía visual.

El detalle del diseño final de cada pantalla se documenta con capturas en la sección 8.

7. Fase de Desarrollo y Construcción

7.1 Stack tecnológico

Tecnologías del backend:

Tecnología	Versión	Rol
FastAPI	0.104.1	Framework web / API REST
SQLAlchemy	2.0.23	ORM de acceso a datos
PostgreSQL (psycopg2)	2.9.9	Base de datos relacional
Pydantic	2.5.0	Validación de datos y esquemas
PyJWT	2.8.1	Tokens de autenticación
bcrypt	4.1.1	Cifrado de contraseñas
slowapi	0.1.9	Limitación de tasa
google-genai	0.3.0	Integración con Gemini (IA)
OpenCV (headless)	4.13.0	Visión por computador (OMR)
NumPy	2.4.4	Cálculo numérico (OMR)
PyMuPDF	1.27.2	PDF ↔ imagen y plantilla OMR
pandas / openpyxl	2.1.3 / 3.1.2	Exportación a Excel

Tabla 7. Componentes del stack del backend.

Tecnologías del frontend:

Tecnología	Versión	Rol
Next.js	16.2.2	Framework React (App Router)
React	19.2.4	Librería de interfaz
Tailwind CSS	v4	Estilos utilitarios
recharts	3.8.1	Gráficas de datos
lucide-react	1.7.0	Iconografía
next-themes	0.4.6	Modo claro/oscuro
lottie-react	2.4.1	Animaciones (demo)

Tabla 8. Componentes del stack del frontend.

7.2 Estructura del proyecto

```
Software/
├── edu-grade-mocavi/           # BACKEND (FastAPI)
│   ├── app/
│   │   ├── main.py             # endpoints de la API (~30 rutas)
│   │   ├── models.py          # entidades SQLAlchemy (modelo de datos)
│   │   ├── schemas.py         # esquemas Pydantic (validación)
│   │   ├── auth.py            # JWT + bcrypt + dependencias de rol
│   │   ├── omr.py             # motor de visión por computador (OMR)
│   │   └── database.py         # conexión y sesión de base de datos
│   └── frontend/              # FRONTEND (Next.js)
│       ├── src/
│       │   ├── app/           # rutas (App Router)
│       │   │   ├── page.js    # landing
│       │   │   ├── login/     # login + portal docente
│       │   │   └── dashboard/estudiante/ # portal del estudiante
│       │   ├── components/    # componentes reutilizables (HelpTip, charts...)
│       │   └── lib/api.js      # cliente HTTP central (JWT, errores, 401)
```

7.3 Construcción del backend

El backend centraliza la lógica de negocio. Un ejemplo representativo es el endpoint que califica hojas con OMR: lee las marcas, identifica al estudiante por su nombre mediante IA y persiste el resultado en el periodo elegido por el docente, de modo que aparezca en el detalle del estudiante.

```
@app.post("/upload-exams-omr")
@limiter.limit("30/minute")
async def upload_exams_omr(file: UploadFile, answer_key: str = Form(...),
                           grade=Form(None), grupo=Form(None),
                           period_id=Form(None), ...):
    pages = await asyncio.to_thread(omr.read_marks, content, ct, num_q, 4)
    roster = db.query(models.Student).filter_by(grade=grade, grupo=grupo).all()
    # Identificación automática del estudiante por el NOMBRE manuscrito (IA de
    visión)
    identified = await _identify_names_by_ai([p["name_img"] for p in pages],
    roster)
    for idx, pg in enumerate(pages):
        score = round(correct / num_q * 4 + 1, 2)
        matched = identified[idx]
        if matched:
            # se persiste para el detalle del
            estudiante
            db.add(models.ExamResult(exam_id=db_exam.id, student_id=matched.id,
            answers=answers_detail, score=score))
```

7.4 Construcción del frontend

Toda la comunicación con el backend pasa por un cliente HTTP central (api.js) que inyecta el token JWT, normaliza los errores y dispara el cierre de sesión automático ante un 401. Esto evita repetir lógica de red en cada componente y unifica el manejo de errores.

```
async function request(path, { method = "GET", body, headers = {}, isForm =
false } = {}) {
    const finalHeaders = { ...headers };
```

```
const token = getToken();
if (token) finalHeaders.Authorization = `Bearer ${token}`;    // JWT automático
if (body !== undefined && !isForm) {
  finalHeaders["Content-Type"] = "application/json";
  body = JSON.stringify(body);
}
const res = await fetch(`${BASE}${path}`, { method, headers: finalHeaders,
body });
if (res.status === 401) { emitUnauthorized(); throw new ApiError("Sesión
expirada", {status:401}); }
const data = await res.json().catch(() => null);
if (!res.ok) throw new ApiError(data?.detail || "Error", { status: res.status,
data });
return data;
}
```

8. Resultados: interfaz construida

Esta sección presenta la interfaz final del sistema mediante capturas tomadas directamente de la aplicación en ejecución, organizadas por portal.

8.1 Sitio público



Figura 1. Página de inicio (landing): propuesta de valor y suscripción al boletín «Mantente al día».

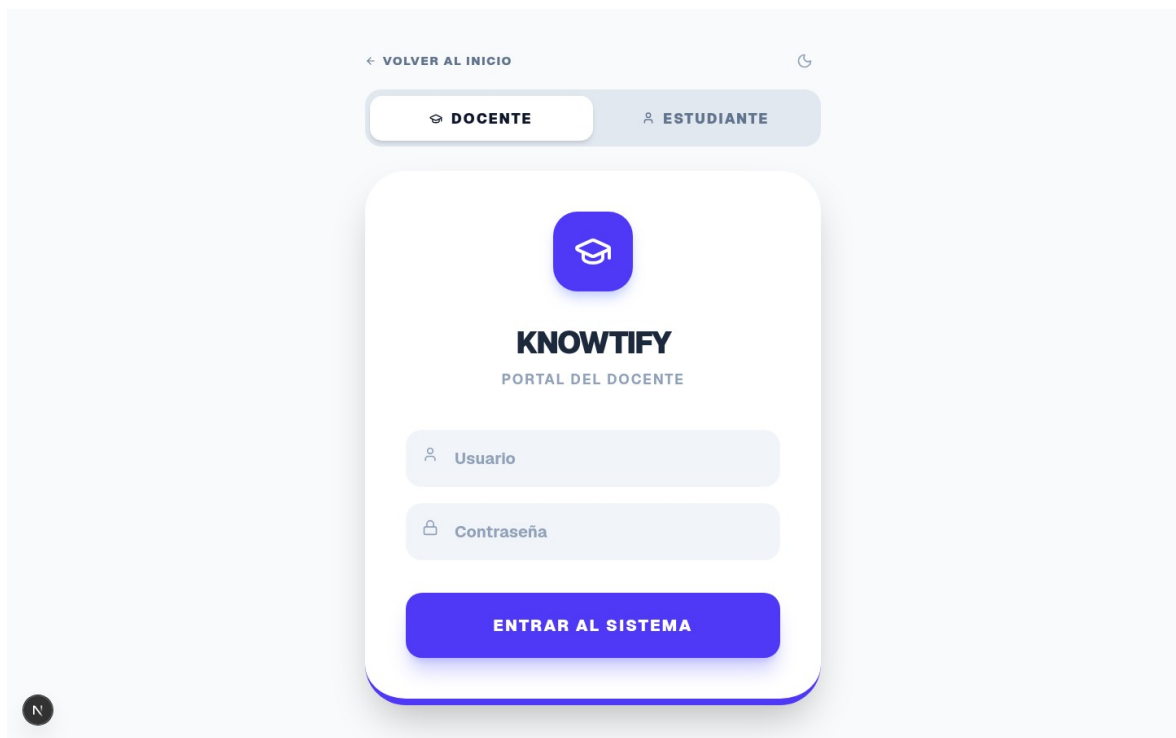


Figura 2. Inicio de sesión — Portal del Docente (usuario y contraseña).

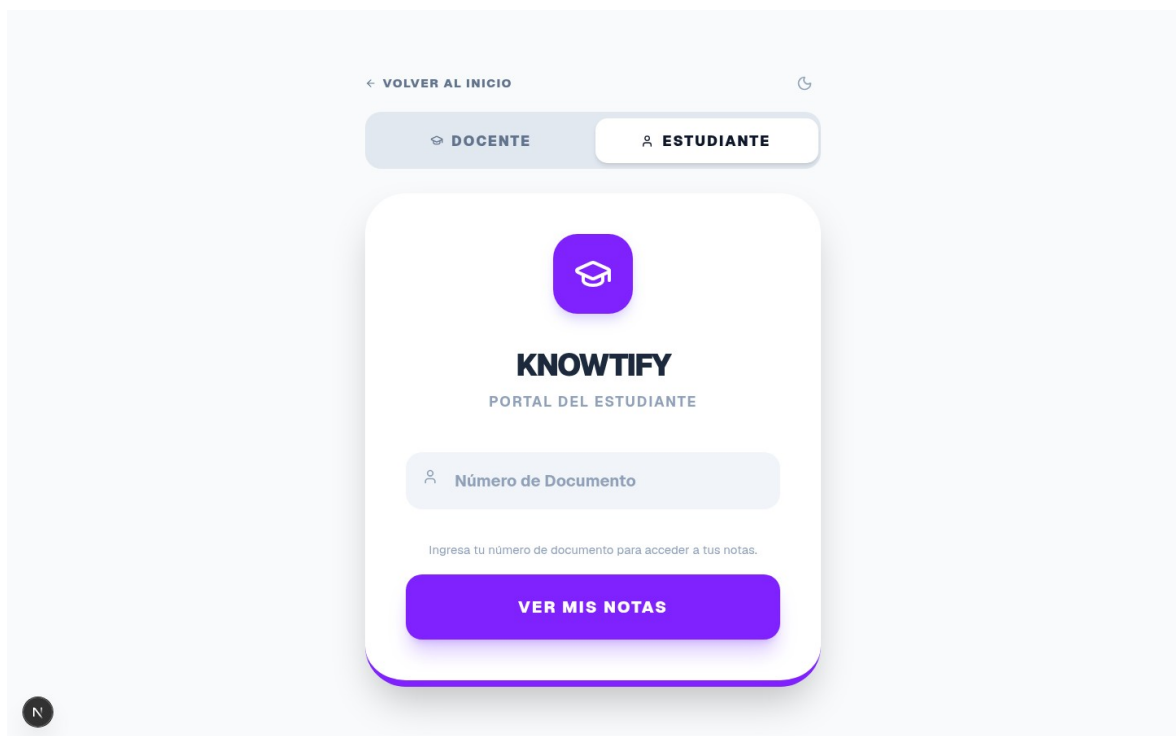


Figura 3. Inicio de sesión — Portal del Estudiante (número de documento).

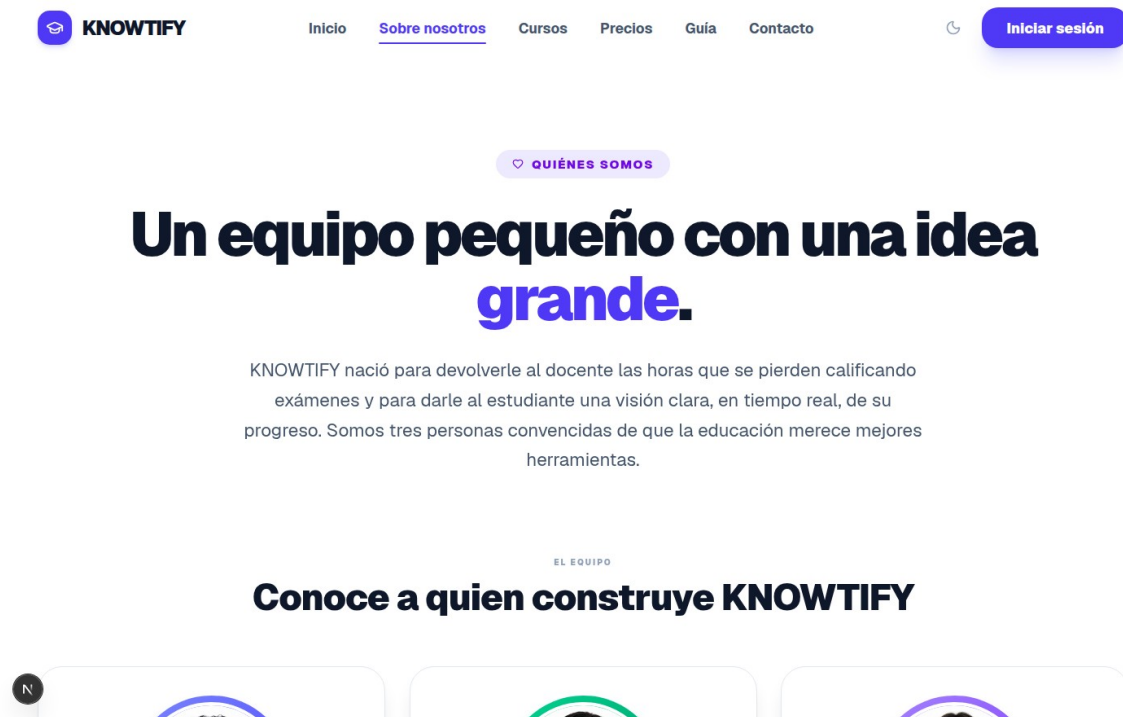


Figura 4. Sección institucional «Sobre nosotros».



Figura 5. Sección «Cursos».

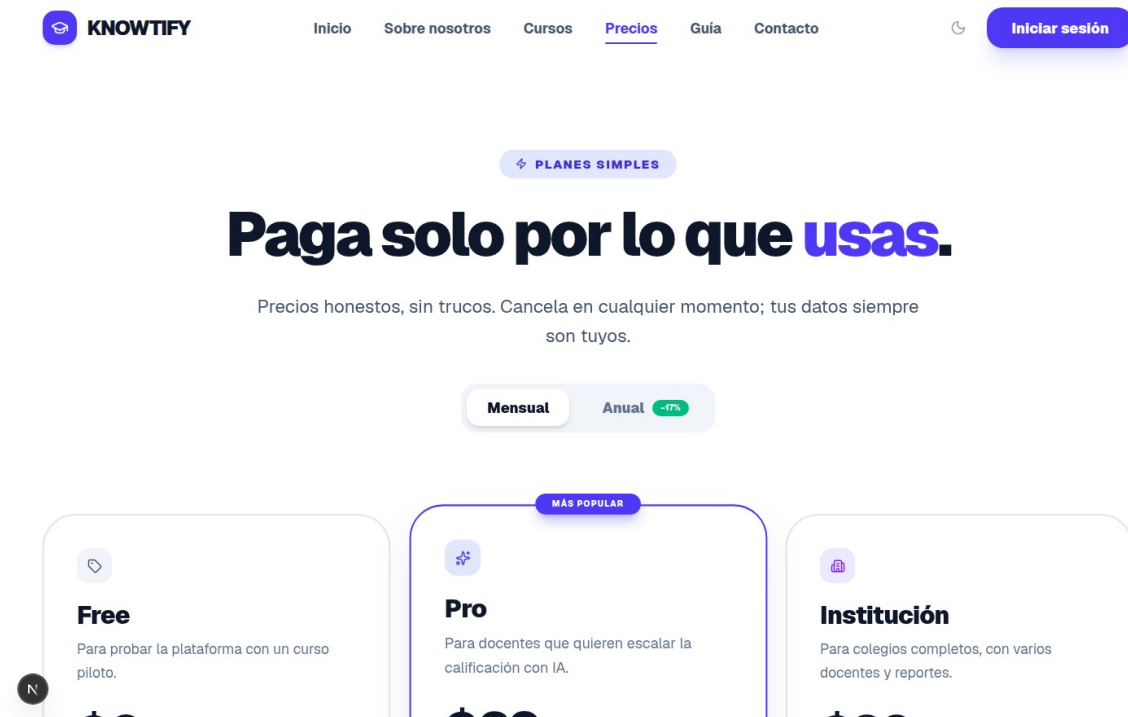


Figura 6. Sección «Precios».



Figura 7. Sección «Guía» de uso de la plataforma.

 **KNOWTIFY**

[Inicio](#) [Sobre nosotros](#) [Cursos](#) [Precios](#) [Guía](#) [Contacto](#)

 [Iniciar sesión](#)

[ESTAMOS CERCA](#)

Hablemos de tu aula.

Sea para una demo, una consulta o un comentario, contesto siempre. Escribenos y te respondemos en menos de 24 horas.

 EMAIL
developers@knowtify.app

 TELÉFONO
+57 302 557 6960

 UBICACIÓN
Montería, Colombia

Envíanos un mensaje

Te respondemos al correo que indiques.

NOMBRE

EMAIL

Tu nombre

tu@correo.com

MOTIVO

Consulta general

Figura 8. Sección «Contacto».

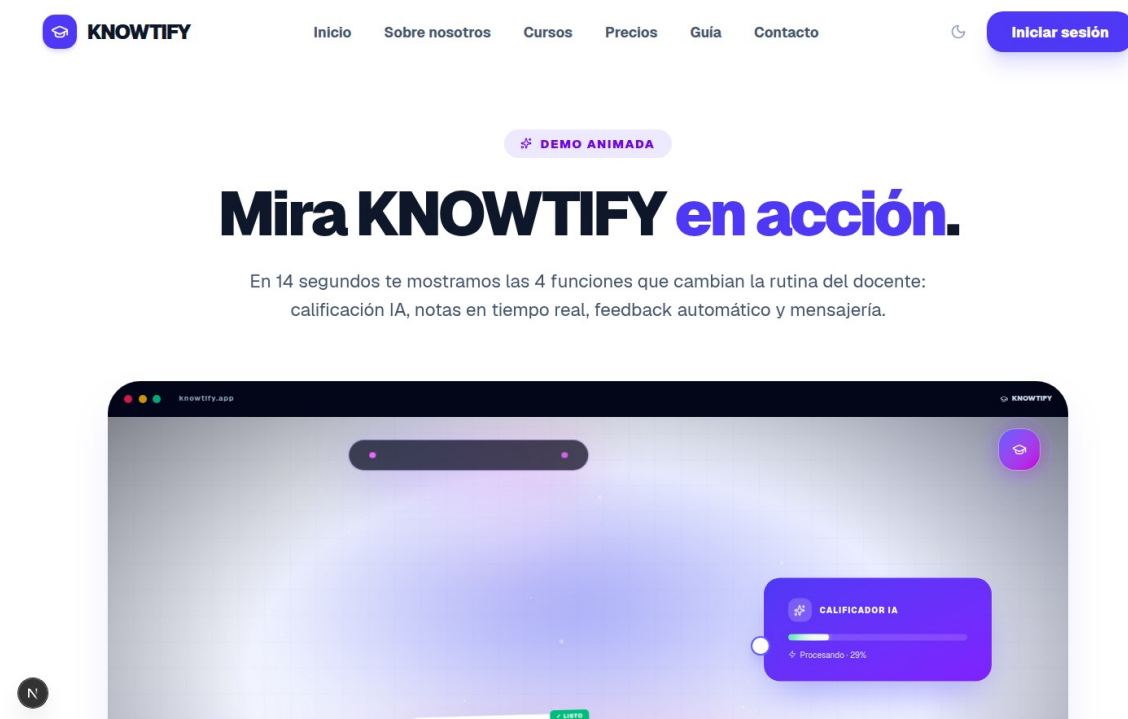


Figura 9. Demostración animada del flujo de calificación.

8.2 Portal del Docente

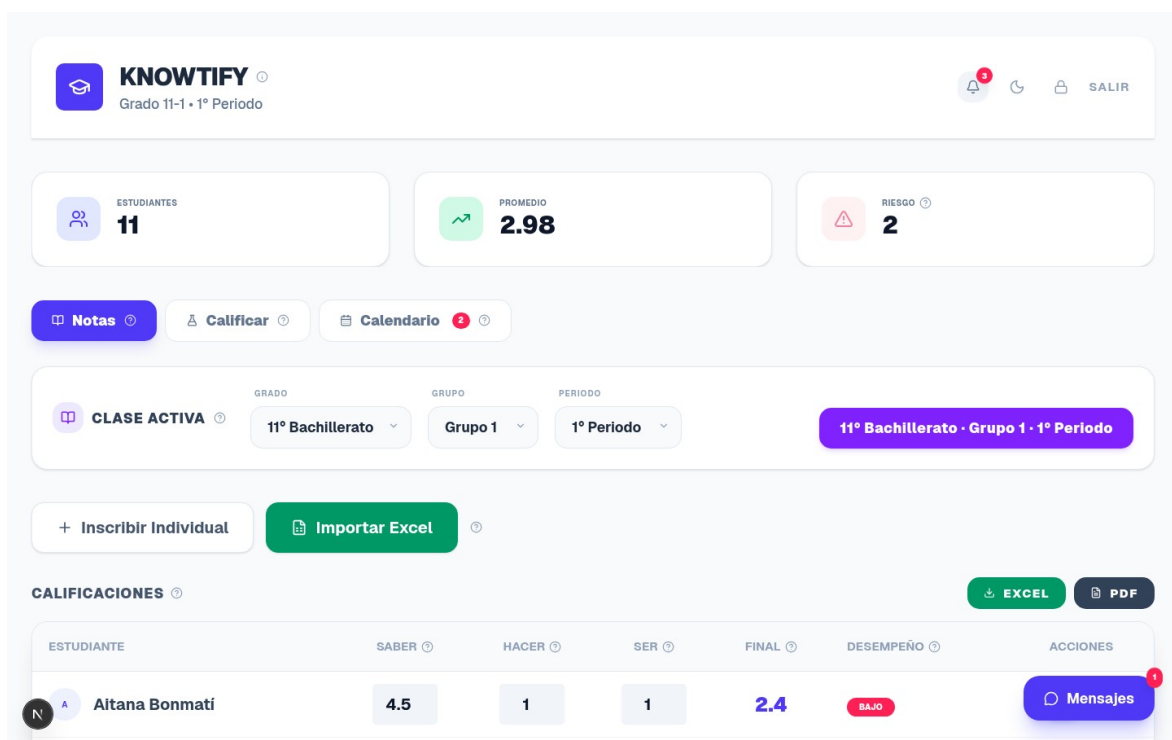


Figura 10. Pestaña «Notas»: tablero de calificaciones por dimensión (Saber/Hacer/Ser), promedio y riesgo.

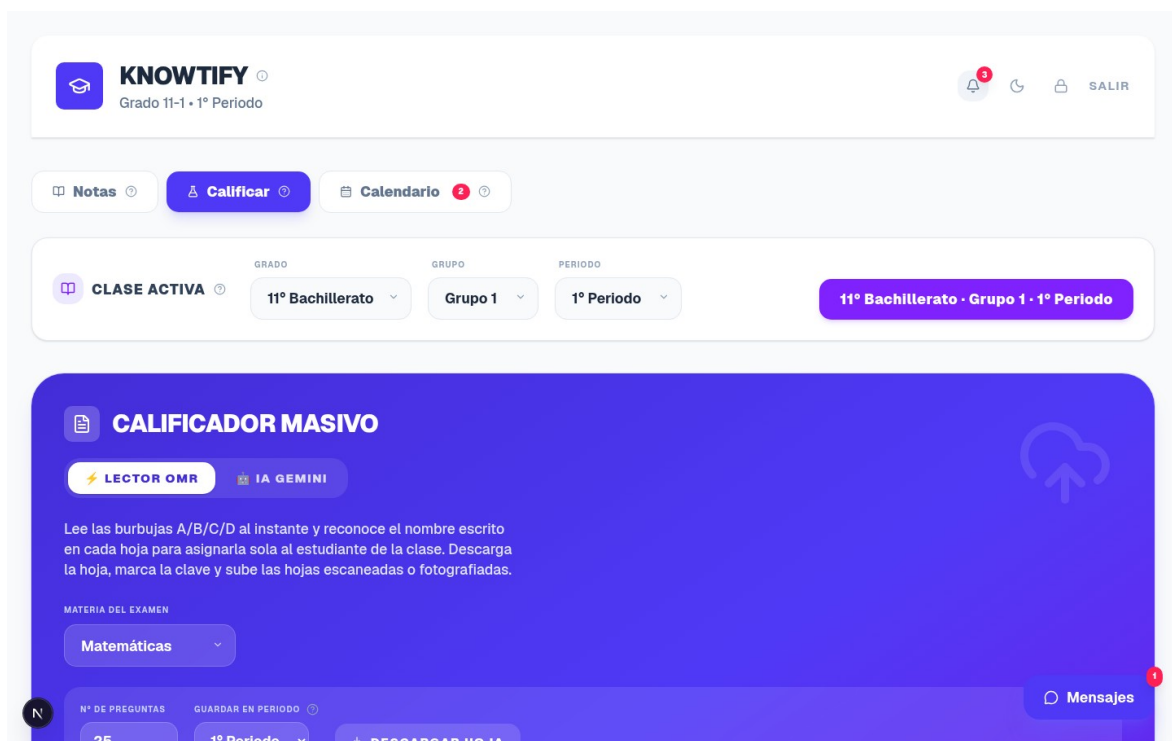


Figura 11. Pestaña «Calificar»: Calificador Masivo con Lector OMR e IA Gemini, selección de periodo y materia.

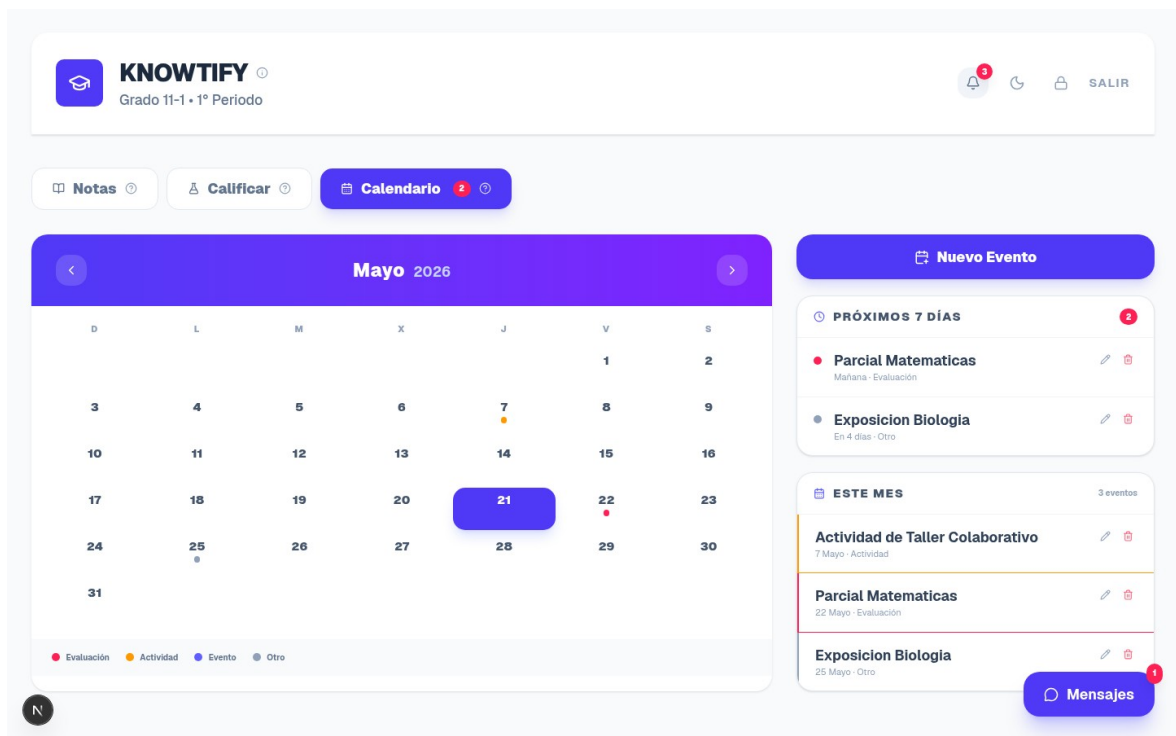


Figura 12. Pestaña «Calendario»: gestión de eventos académicos por grado y grupo.

8.3 Portal del Estudiante

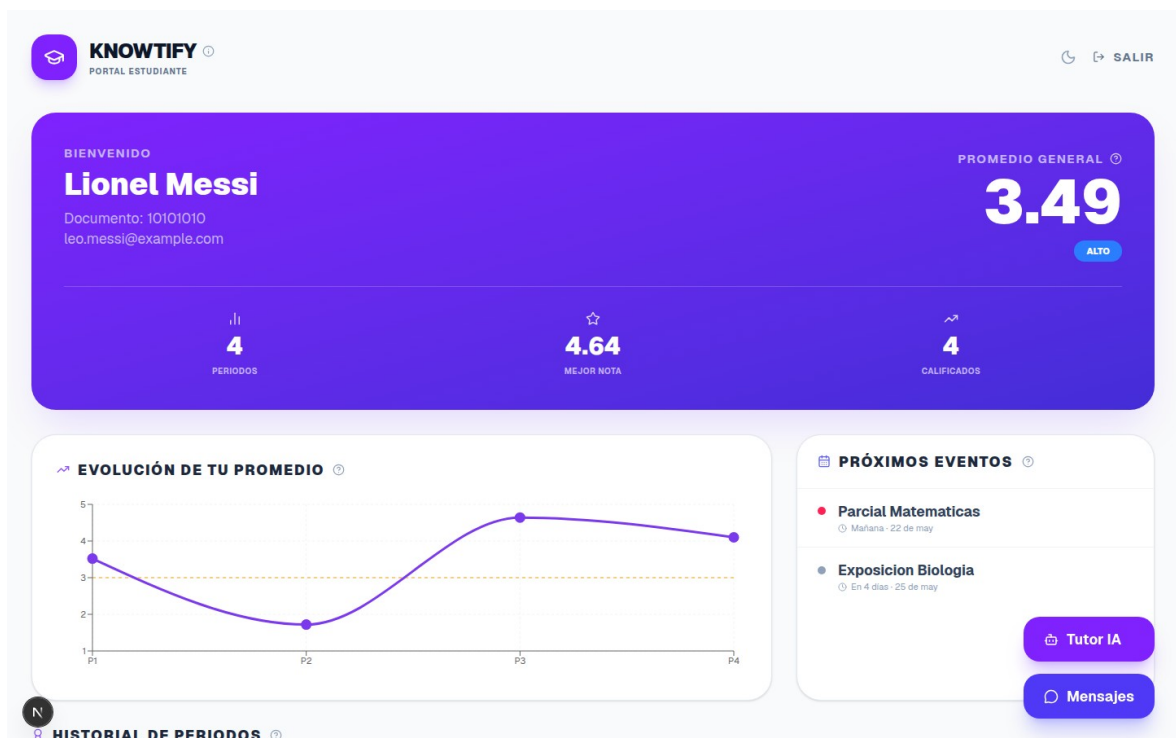


Figura 13. Tablero del estudiante: promedio general, evolución del promedio por periodo y próximos eventos.

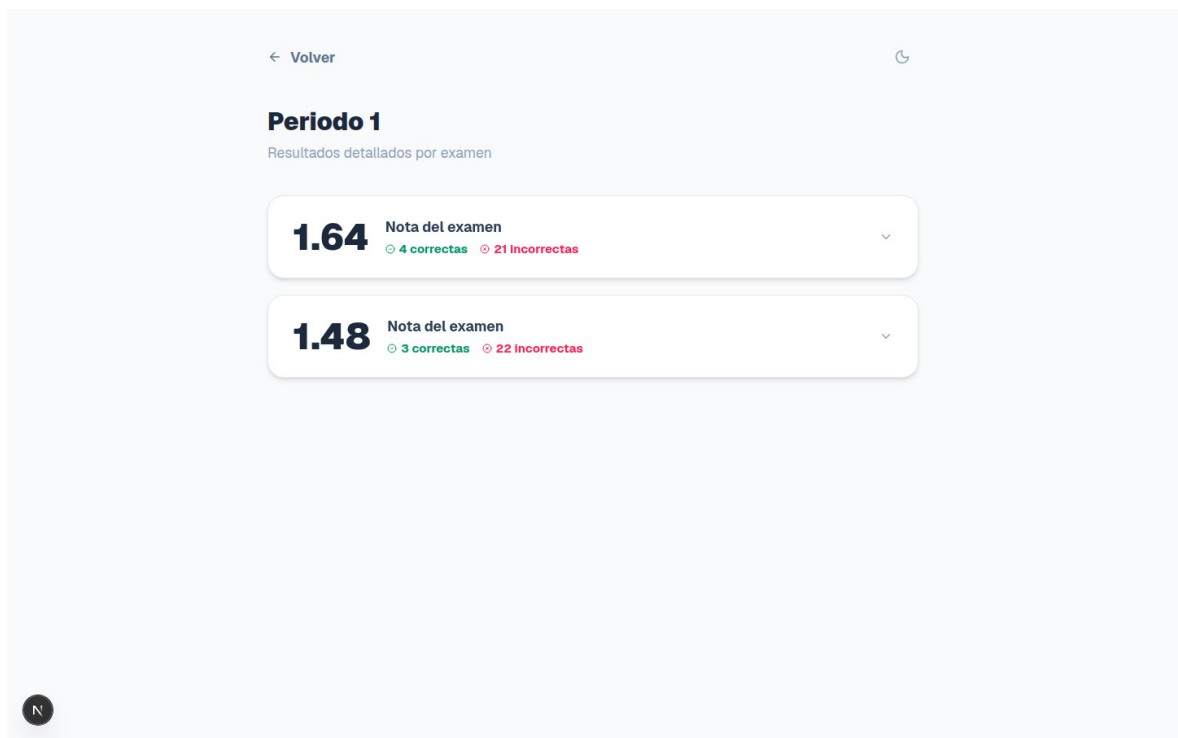


Figura 14. Detalle por periodo: exámenes calificados con OMR que aparecen automáticamente en el periodo.

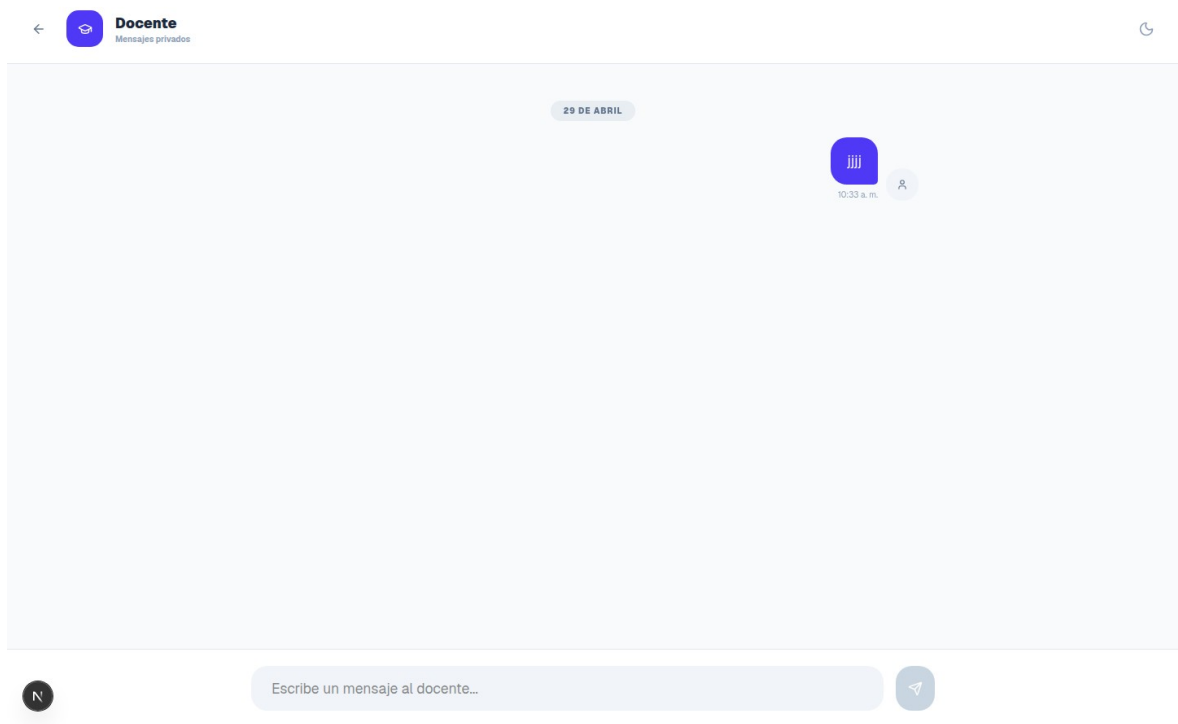


Figura 15. Mensajería con el docente en tiempo real.

9. Pruebas y verificación

Cada funcionalidad se verificó de extremo a extremo (E2E) contra el sistema en ejecución. Las pruebas más relevantes del módulo de calificación fueron:

Caso de prueba	Resultado esperado	Estado
Lectura OMR de una hoja	Lee 25/25 respuestas y calcula la nota.	✓ Correcto
Identificación por nombre (exacto)	Asigna la hoja al estudiante correcto.	✓ Correcto
Identificación por nombre (con error/tilde)	Empareja pese a minúsculas/acentos.	✓ Correcto
Guardado en el periodo elegido	El examen aparece en el detalle del periodo.	✓ Correcto
Aislamiento entre periodos	El examen no aparece en otros periodos.	✓ Correcto
Suscripción al boletín	Alta, duplicado y correo inválido controlados.	✓ Correcto

Tabla 9. Resumen de pruebas E2E del módulo de calificación y boletín.

10. Cronología del desarrollo

El historial de versiones (control de versiones Git) refleja los incrementos principales del proyecto:

Fecha	Hito
30/04/2026	Estado base v2.1 e instrucciones de reversión (RESTORE.md).
20/05/2026	Calificador OMR (OpenCV), landing y secciones, demo animada y despliegue.
21/05/2026	Reorganización del portal docente (Notas/Calificar/Calendario) y ayuda contextual.
21/05/2026	Identificación por nombre con IA, selección de periodo y boletín «Mantente al día».

Tabla 10. Hitos principales del desarrollo (historial Git).

11. Conclusiones

El proyecto KNOWTIFY cumplió el objetivo de construir una plataforma educativa full-stack que automatiza la calificación y centraliza la gestión académica. La fase de diseño fue decisiva: la arquitectura de tres capas, el modelo de datos basado en entidades claras y el diseño del motor OMR alrededor de una geometría única permitieron construir el sistema de forma incremental y verificable.

Entre los logros de diseño más destacados se encuentran la identificación automática del estudiante por nombre mediante IA de visión —que elimina trabajo manual sin sacrificar robustez— y la integración coherente de la calificación con el seguimiento del desempeño del estudiante. El sistema queda preparado para crecer con nuevos módulos gracias al desacople entre frontend y backend y al cliente HTTP central.

— *Fin del informe* —